Overview of Statistical Learning and Modeling – 36-600

Lab 6T: Variable Selection
Due Wednesday, October 1, 11:59pm

Trishla Nair

Data

We'll begin by importing the heart-disease dataset and log-transforming the response variable, `Cost`. Also, so that the dataset "plays well" with our `bestglm` later, we will change the name `Cost` to `y` and put `y` last among the columns (make sure you see how the code below does so).

```
suppressMessages(library(tidyverse))
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
df      <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/heart_d
df      <- df[, -10]
w       <- which(df$Cost > 0)
df      <- df[w, ]
df$Cost <- log(df$Cost)
df$y    <- df$Cost   # create a new column on the fly
df %>% select(., -Cost) -> df
summary(df)
```

```
##       Age           Gender    Interventions        Drugs
##  Min.   :24.00   Female:608   Min.   : 0.000   Min.   :0.0000
##  1st Qu.:55.00   Male  :177   1st Qu.: 1.000   1st Qu.:0.0000
##  Median :60.00                Median : 3.000   Median :0.0000
##  Mean   :58.73                Mean   : 4.722   Mean   :0.3389
##  3rd Qu.:64.00                3rd Qu.: 6.000   3rd Qu.:0.0000
##  Max.   :70.00                Max.   :47.000   Max.   :2.0000
##     ERVisit       Complications     Comorbidities       Duration
##  Min.   : 0.000   Min.   :0.00000   Min.   : 0.000   Min.   :  0.0
##  1st Qu.: 2.000   1st Qu.:0.00000   1st Qu.: 0.000   1st Qu.: 42.0
##  Median : 3.000   Median :0.00000   Median : 1.000   Median :167.0
##  Mean   : 3.424   Mean   :0.05478   Mean   : 3.781   Mean   :164.7
##  3rd Qu.: 5.000   3rd Qu.:0.00000   3rd Qu.: 5.000   3rd Qu.:281.0
##  Max.   :20.000   Max.   :1.00000   Max.   :60.000   Max.   :372.0
##         y
##  Min.   : 0.470
##  1st Qu.: 5.090
##  Median : 6.235
##  Mean   : 6.314
##  3rd Qu.: 7.582
##  Max.   :10.872
```

Question 1

Split the data into training and test sets. Call these objects `df.train` and `df.test`. Assume that 70% of the data will be used to train the linear regression model. Recall that

```
s <- sample(nrow(df), round(0.7 * nrow(df)))
```

will randomly select 70% of the rows for training. Also recall that

```
df[s, ] and df[-s, ]
```

are ways of filtering the data frame into the training set and the test set, respectively. (Remember to set the random number seed!)

```
# FILL ME IN WITH CODE
set.seed(42)
s <- sample(nrow(df), round(0.7 * nrow(df)))
df.train<-df[s, ]
df.test<-df[-s, ]
```

Question 2

Perform a multiple linear regression analysis, regressing `y` upon all the other variables, and compute the resulting mean-squared error. Also print out the *adjusted*-R^2 value; if you assigned the output of your linear regression function call to `lm.out`, then what you'd print out is `summary(lm.out)$adj.r.squared`

```
# FILL ME IN WITH CODE
lm.out <- lm(y ~ ., data = df.train)
summary(lm.out)
```

```
##
## Call:
## lm(formula = y ~ ., data = df.train)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.4208 -0.6677  0.0198  0.6291  3.2587
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)    5.0786025  0.4312742  11.776  < 2e-16 ***
## Age           -0.0098058  0.0073017  -1.343  0.17985
## GenderMale    -0.0957881  0.1192281  -0.803  0.42210
## Interventions  0.1845922  0.0093822  19.675  < 2e-16 ***
## Drugs         -0.0524284  0.0874068  -0.600  0.54888
## ERVisit        0.0804332  0.0226224   3.555  0.00041 ***
## Complications  0.8750535  0.2063197   4.241 2.61e-05 ***
## Comorbidities  0.0470264  0.0100271   4.690 3.46e-06 ***
## Duration       0.0028354  0.0004788   5.922 5.65e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 1.161 on 541 degrees of freedom
## Multiple R-squared:  0.6151, Adjusted R-squared:  0.6094
## F-statistic: 108.1 on 8 and 541 DF,  p-value: < 2.2e-16
```

```
Cost.pred <- predict(lm.out, newdata = df.test)
```

Question 3

Install the `bestglm` package, if you do not have it installed already. Then load that library and use the function `bestglm()` to perform best subset selection on the training data. Use both AIC and BIC, and for each, display the best model. How many predictor variables are retained in the best models? (Don't count the intercepts.) Do the relative numbers of variables abide by your expectations? Is one model a subset of the other? (*Hint*: see the documentation for `bestglm()` and look at the part under "Value" – this describes the `R` object that `bestglm()` returns. The best model is included as one element contained within that object. Let `bg.bic` be your output from `bestglm()` for BIC, and `bg.aic` be the output for AIC. If the documentation states that `xx` is the element of the output that contains the best model, then simply print, e.g., `bg.bic$xx`. In the end, what usually gets returned from a function call will either be a vector or a list – in this case it will be a list. If you need to know the names of the elements of the list, type, e.g., `names(bg.bic)`. Doing that here might be helpful: the element with the best model might jump out at you!)

```
# FILL ME IN WITH CODE
library(bestglm)
```

```
## Warning: package 'bestglm' was built under R version 4.4.3
```

```
## Loading required package: leaps
```

```
## Warning: package 'leaps' was built under R version 4.4.3
```

```
mse.full <- mean((predict(lm.out, newdata = df.test) - df.test$y)^2)
bg.bic<- bestglm(df.train, family = gaussian, IC = "AIC")
```

```
## binary categorical variables converted to 0-1 so 'leaps' could be used.
```

```
print(bg.bic$BestModel)
```

```
##
## Call:
## lm(formula = y ~ ., data = data.frame(Xy[, c(bestset[-1], FALSE),
##     drop = FALSE], y = y))
##
## Coefficients:
##   (Intercept)  Interventions        ERVisit  Complications  Comorbidities
##      4.502783       0.185368       0.071525       0.886273       0.047401
##      Duration
##      0.002761
```

```r
bg.aic<- bestglm(df.train, family = gaussian, IC = "AIC")
```

```
## binary categorical variables converted to 0-1 so 'leaps' could be used.
```

```r
print(bg.aic$BestModel)
```

```
##
## Call:
## lm(formula = y ~ ., data = data.frame(Xy[, c(bestset[-1], FALSE),
##     drop = FALSE], y = y))
##
## Coefficients:
##   (Intercept)  Interventions        ERVisit  Complications  Comorbidities
##      4.502783       0.185368       0.071525       0.886273       0.047401
##      Duration
##      0.002761
```

There are 6 variables retained in the best model. The relative numbers do not exactly abide by my expec

Question 4

The output of `bestglm()` contains, as you saw above, a "best model". According to the documentation for `bestglm()`, this list element is an "`lm`-object representing the best fitted algorithm." That means you can pass it to `predict()` in order to generate predicted response values (where the response is in the `y` column of your data frames). Given this information: generate mean-squared error values for the BIC- and AIC-selected models. Are these values larger or smaller than the value you got for linear regression?

```r
# FILL ME IN WITH CODE
mse.bic <- mean((predict(bg.bic$BestModel, newdata = df.test) - df.test$y)^2)

mse.aic <- mean((predict(bg.aic$BestModel, newdata = df.test) - df.test$y)^2)

cat("Full model MSE:", mse.full, "\n")
```

```
## Full model MSE: 1.719453
```

```r
cat("BIC-selected model MSE:", mse.bic, "\n")
```

```
## BIC-selected model MSE: 1.714698
```

```r
cat("AIC-selected model MSE:", mse.aic, "\n")
```

```
## AIC-selected model MSE: 1.714698
```

Based on the values I got, these values appear smaller compares to what I got from linear regression.

Here is code that allows you to visualize, e.g., the BIC as a function of number of variables. Note that in this example, `bg.bic` would be the output of `bestglm(..., IC = "BIC")` (you'll want to remove `eval=FALSE` for this code chunk to run in the process of creating your document, that is, once your `bg.bic` object exists). This is just *FYI*: if you ever use variable selection in practice, you might find this visualization useful.

```
bic    <- bg.bic$Subsets["AIC"]
df.bic <- data.frame("p" = 1:ncol(df.train) - 1, "BIC" = bic[, 1])

g <- ggplot(data = df.bic, mapping = aes(x = p, y = BIC)) +
       geom_point(size = 1.5, color = "blue") +
       geom_line(color = "blue") +
       ylim(min(bic), min(bic + 100))    # a quick and dirty way to try to zero in on the
                                         #  correct range to see the minimum
suppressWarnings(print(g))               # a way to get around pesky ggplot warnings
```

Question 5

Run the `summary()` function with the best BIC model from above. This produces output akin to that of the output from summarizing a linear model (e.g., one output by `lm()`). What is the adjusted R^2 value? What does the value imply about the quality of the linear fit with the best subset of variables?

```
summary(bg.bic$BestModel)
```

```
##
## Call:
## lm(formula = y ~ ., data = data.frame(Xy[, c(bestset[-1], FALSE),
##     drop = FALSE], y = y))
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.5446 -0.6506 -0.0033  0.6357  3.4407
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   4.5027833  0.0989437  45.509  < 2e-16 ***
## Interventions 0.1853681  0.0093494  19.827  < 2e-16 ***
## ERVisit       0.0715250  0.0201992   3.541 0.000433 ***
## Complications 0.8862726  0.2060870   4.300 2.02e-05 ***
## Comorbidities 0.0474014  0.0099715   4.754 2.56e-06 ***
## Duration      0.0027607  0.0004764   5.795 1.16e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.161 on 544 degrees of freedom
## Multiple R-squared:  0.6131, Adjusted R-squared:  0.6096
## F-statistic: 172.4 on 5 and 544 DF,  p-value: < 2.2e-16
```

The adjusted R^2 is 0.6096. It implies that when adjusted, the data is some what better fitted to the li